6.3. PREDICCIÓN DEL INTERBLOQUEO

Una estrategia para resolver el problema del interbloqueo que difiere sutilmente de la prevención del interbloqueo es la predicción del interbloqueo[2]. En la **prevención del interbloqueo**, se restringen las solicitudes de recurso para impedir al menos una de las cuatro condiciones de interbloqueo. Esto se realiza o bien indirectamente, impidiendo una de las tres condiciones de gestión necesarias (exclusión mutua, retención y espera, y sin expropiación), o directamente, evitando la espera circular. Esto conlleva un uso ineficiente de los recursos y una ejecución ineficiente de los procesos. La **predicción del interbloqueo**, por otro lado, permite las tres condiciones necesarias pero toma decisiones razonables para asegurarse de que nunca se alcanza el punto del interbloqueo. De esta manera, la predicción permite más concurrencia que la prevención. Con la predicción del interbloqueo, se decide dinámicamente si la petición actual de reserva de un recurso, si se concede, podrá potencialmente causar un interbloqueo. La predicción del interbloqueo, por tanto, requiere el conocimiento de las futuras solicitudes de recursos del proceso.

En esta sección se describen dos técnicas para predecir el interbloqueo:

- No iniciar un proceso si sus demandas podrían llevar al interbloqueo.
- No conceder una petición adicional de un recurso por parte de un proceso si esta asignación podría provocar un interbloqueo.

DENEGACIÓN DE LA INICIACIÓN DEL PROCESO

Considere un sistema con n procesos y m tipos diferentes de recursos. En el mismo se definen los siguientes vectores y matrices:

Recursos = $\mathbf{R} = (R_1, R_2, ..., R_m)$	cantidad total de cada recurso en el sistema
Disponibles = $\mathbf{D} = (D_1, D_2, ..., D_m)$	cantidad total de cada recurso no asignada a ningún proceso
Necesidad = $\mathbf{N} = \begin{vmatrix} N_{11} & N_{12} & ... & N_{1m} \\ N_{21} & N_{22} & ... & N_{2m} \\ . & . & . & . \\ . & . & . & . \\ . & . & . & . \\ N_{n1} & N_{n2} & ... & N_{nm} \end{vmatrix}$	N_{ij} = necesidades del proceso i con respecto al recurso j
Asignación = $\mathbf{A} = \begin{vmatrix} A_{11} & A_{12} & ... & A_{1m} \\ A_{21} & A_{22} & ... & A_{2m} \\ . & . & . & . \\ . & . & . & . \\ . & . & . & . \\ A_{n1} & A_{n2} & ... & A_{nm} \end{vmatrix}$	A_{ij} = asignación actual al proceso i con respecto al recurso j

[2] El término *predicción* es un poco confuso. De hecho, se podrían considerar las estrategias estudiadas en esta sección como ejemplos de prevención del interbloqueo debido a que impiden efectivamente la aparición del interbloqueo.

La matriz de Necesidad proporciona los requisitos máximos de cada proceso con respecto a cada recurso, estando una fila dedicada a cada proceso. Esta información debe declararse con antelación por el proceso para que la predicción del interbloqueo funcione. De modo similar, la matriz de Asignación proporciona la asignación actual a cada proceso. Se cumplen las siguientes relaciones:

1. $R_j = D_j + \sum_{i=1}^{n} A_{ij}$, para todo j	Todos los recursos están disponibles o asignados.
2. $N_{ij} \leq R_j$, para todo i, j	Ningún proceso puede necesitar más de la cantidad total de recursos existentes en el sistema.
3. $A_{ij} \leq N_{ij}$, para todo i, j	Ningún proceso tiene asignados más recursos de cualquier tipo que sus necesidades originales de ese recurso.

Con estas cantidades definidas, se puede establecer una política de predicción del interbloqueo que rechace iniciar un nuevo proceso si sus requisitos de recursos pudiesen conducir al interbloqueo. Se inicia un nuevo proceso P_{n+1} sólo si:

$$R_j \geq N_{(n+1)j} + \sum_{i=1}^{n} N_{ij} \qquad \text{para todo } j$$

Es decir, sólo puede iniciarse un proceso si se pueden satisfacer las necesidades máximas de todos los procesos actuales más las del nuevo proceso. Esta estrategia está lejos de ser óptima, debido a que asume el peor caso: todos los procesos solicitarán sus necesidades máximas simultáneamente.

DENEGACIÓN DE ASIGNACIÓN DE RECURSOS

La estrategia de la denegación de asignación de recursos, denominada algoritmo del banquero[3], se propuso por primera vez en [DIJK65]. Se comenzará definiendo los conceptos de estado y de estado seguro. Considere un sistema con un número fijo de procesos y de recursos. En un determinado momento un proceso puede tener cero o más recursos asignados. El **estado** del sistema refleja la asignación actual de recursos a procesos. Por tanto, el estado consiste en los dos vectores, Recursos y Disponibles, y las dos matrices, Necesidad y Asignación, definidas anteriormente. Un **estado seguro** es aquél en el que hay al menos una secuencia de asignación de recursos a los procesos que no implica un interbloqueo (es decir, todos los procesos pueden ejecutarse al completo). Un **estado inseguro** es, evidentemente, un estado que no es seguro.

El siguiente ejemplo muestra estos conceptos. La Figura 6.7a muestra el estado de un sistema que consta de cuatro procesos y tres recursos. La cantidad total de recursos R1, R2, y R3 son de 9, 3 y 6 unidades, respectivamente. En el estado actual se han realizado asignaciones a los cuatro procesos, dejando disponibles 1 unidad de R2 y 1 unidad de R3. La pregunta es: ¿es un estado seguro? Para responder a esta cuestión, se plantea una pregunta intermedia: ¿alguno de los cuatro procesos

[3] Dijkstra usó este nombre debido a la analogía de este problema con uno que se da en la banca, donde los clientes que desean un préstamo de dinero se corresponden con los procesos y el dinero para prestar se corresponde con los recursos. Enunciado como un problema bancario, el problema sería como se especifica a continuación. El banco tiene una reserva limitada de dinero para prestar y una lista de clientes, cada uno con una línea de crédito. En un momento dado, un cliente puede optar por solicitar un préstamo de una cantidad que corresponda con una parte de su línea de crédito, y no hay garantía de que haga un reembolso hasta después de haber obtenido la cantidad máxima del préstamo. El banquero puede rechazar un préstamo a un cliente si hay riesgo de que el banco se quede sin fondos suficientes para satisfacer solicitudes de préstamo posteriores que permitirán que los clientes realicen finalmente un reembolso.

puede ejecutarse por completo con los recursos disponibles? Es decir, ¿puede satisfacerse con los recursos disponibles la diferencia entre los requisitos máximos y la asignación actual de algún proceso?

En términos de las matrices y vectores presentados previamente, la condición que debe cumplirse para el proceso *i* es la siguiente:

$$N_{ij} - A_{ij} \leq D_j, \text{ para todo } j$$

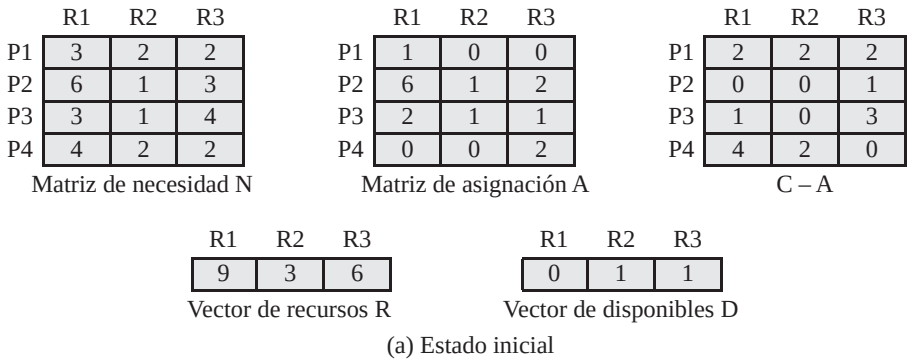
Claramente, esto no es posible en el caso de P1, que tiene sólo una unidad de R1 y requiere 2 unidades adicionales, otras 2 de R2, así como 2 unidades de R3. Sin embargo, asignando una unidad de R3 al proceso P2, éste logra tener asignados sus recursos máximos requeridos y puede ejecutarse por completo. Asuma que esto se lleva a cabo. Cuando se complete P2, podrá retornar sus recursos al conjunto de recursos disponibles. El estado resultante se muestra en la Figura 6.7b. En ese momento, se repetiría la pregunta de si cualquiera de los procesos restantes puede completarse. En este caso, todos los procesos restantes podrían completarse. Supóngase, que se elige P1, asignándole los recursos requeridos, se completa P1, y devuelve todos sus recursos al conjunto de disponibles. El estado resultante se muestra en la Figura 6.7c. A continuación, se puede completar P3, alcanzándose el estado de la Figura 6.7d. Finalmente, se puede completar P4. En ese instante, todos los procesos se han ejecutado por completo. Por tanto, el estado definido en la Figura 6.7a es seguro.

Estos conceptos sugieren la siguiente estrategia de predicción del interbloqueo, que asegura que el sistema de procesos y recursos está siempre en un estado seguro. Cuando un proceso solicite un conjunto de recursos, supóngase que se concede la petición, actualice el estado del sistema en consecuencia, y determine si el resultado es un estado seguro. En caso afirmativo, se concede la petición. En caso contrario, se bloquea el proceso hasta que sea seguro conceder la petición.

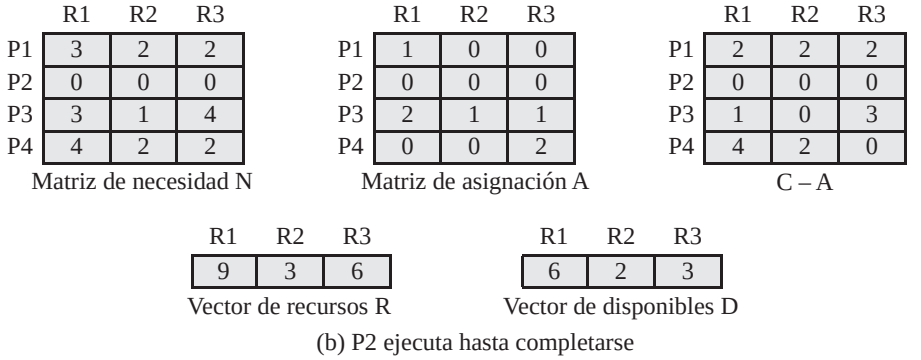
Considérese el estado definido en la Figura 6.8a. Supóngase que P2 solicita una unidad adicional de R1 y otra de R3. Si se asume que se concede la petición, el estado resultante es el de la Figura 6.7a, que ya se ha comprobado previamente que es un estado seguro. Por tanto, es seguro conceder la petición. Retornando al estado de la Figura 6.8a y suponiendo que P1 solicita una unidad de R1 y otra de R3; si se asume que se concede la petición, el estado resultante es el representado en la Figura 6.8b. ¿Es un estado seguro? La respuesta es que no, debido a que cada proceso necesitará al menos una unidad adicional de R1, y no hay ninguna disponible. Por tanto, basándose en la predicción del interbloqueo, la solicitud de P1 se denegaría y P1 se debería bloquear.

Es importante resaltar que la Figura 6.8b no es un estado de interbloqueo. Sólo indica la posibilidad del interbloqueo. Es posible, por ejemplo, que si P1 ejecutara a partir de este estado, liberaría posteriormente una unidad de R1 y una de R3 antes de volver a necesitar estos recursos de nuevo. Si esto sucediera, el sistema retornaría a un estado seguro. Por tanto, la estrategia de predicción de interbloqueo no predice el interbloqueo con certeza; sólo anticipa la posibilidad del interbloqueo y asegura que no haya tal posibilidad.

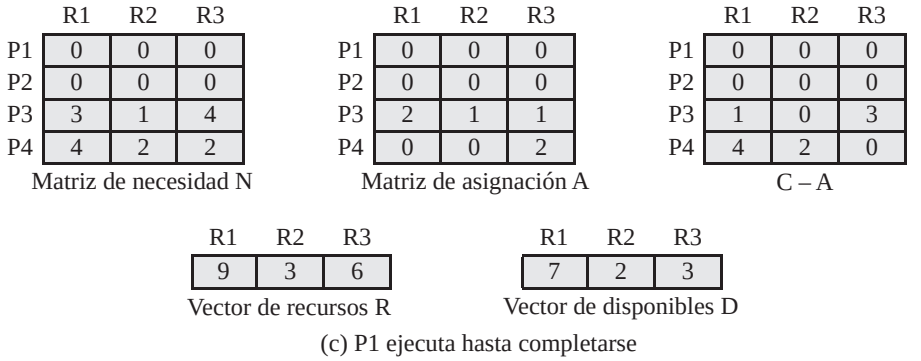
La Figura 6.9 da una versión abstracta de la lógica de predicción del interbloqueo. El algoritmo principal se muestra en la parte (b). La estructura de datos `estado` define el estado del sistema, mientras que `peticion[*]` es un vector que define los recursos solicitados por el proceso *i*. En primer lugar, se hace una prueba para asegurarse de que la petición no excede las necesidades originales del proceso. Si la petición es válida, el siguiente paso es determinar si es posible satisfacerla (es decir, hay suficientes recursos disponibles). Si no es posible, se suspende la ejecución del proceso. En caso de que lo sea, el paso final es determinar si es seguro satisfacer la petición. Para hacer esto, se asignan provisionalmente los recursos al proceso *i* para formar el `nuevo_estado`. A continuación, se realiza una comprobación de si el estado es seguro utilizando el algoritmo de la Figura 6.9c.



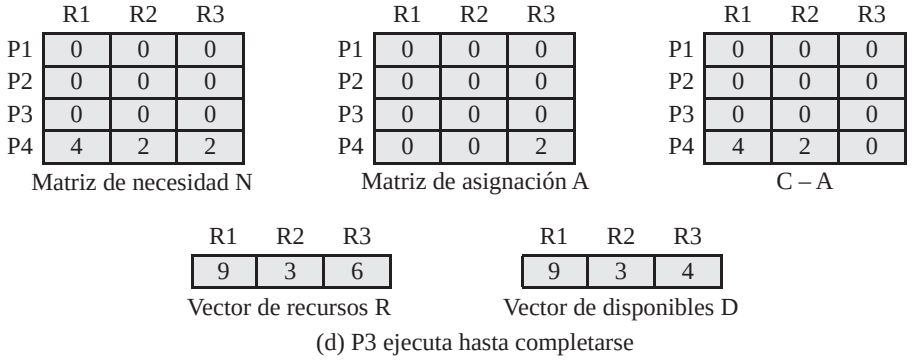
(a) Estado inicial



(b) P2 ejecuta hasta completarse



(c) P1 ejecuta hasta completarse



(d) P3 ejecuta hasta completarse

Figura 6.7. Determinación de un estado seguro.

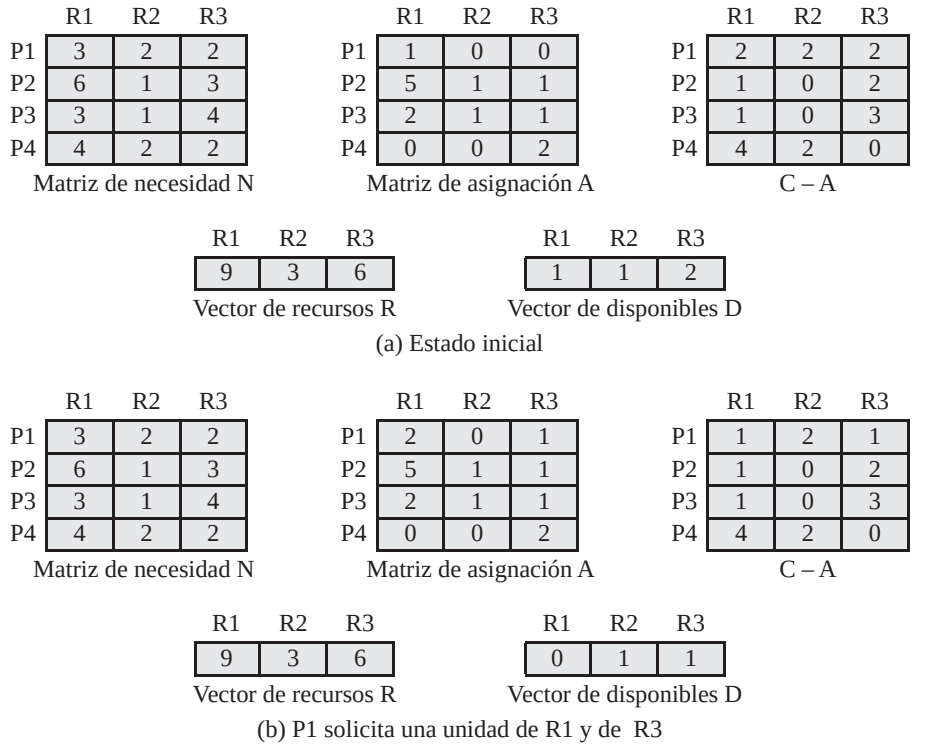


Figura 6.8. Determinación de un estado inseguro.

La predicción del interbloqueo tiene la ventaja de que no es necesario expropiar a los procesos ni retroceder su ejecución, como ocurre con la detección del interbloqueo, y es menos restrictivo que la prevención del interbloqueo. Sin embargo, tiene varias restricciones de uso:

- Deben establecerse por anticipado los requisitos máximos de recursos de cada proceso.
- Los procesos involucrados deben ser independientes, es decir, el orden en el que se ejecutan no debe estar restringido por ningún requisito de sincronización.
- Debe haber un número fijo de recursos que asignar.
- Ningún proceso puede terminar mientras mantenga recursos.

6.4. DETECCIÓN DEL INTERBLOQUEO

Las estrategias de prevención de interbloqueo son muy conservadoras: resuelven el problema del interbloqueo limitando el acceso a los recursos e imponiendo restricciones a los procesos. En el extremo contrario, la estrategia de detección del interbloqueo no limita el acceso a los recursos ni restringe las acciones de los procesos. Con la detección del interbloqueo, los recursos pedidos se conceden a los procesos siempre que sea posible. Periódicamente, el sistema operativo realiza un algoritmo que le permite detectar la condición de espera circular descrita anteriormente en la condición (4) e ilustrada en la Figura 6.6.